JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AsynchronousServerSocketChannel

java.lang.Object
    java.nio.channels.AsynchronousServerSocketChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `AsynchronousChannel`, `Channel`, `NetworkChannel`

```
public abstract class AsynchronousServerSocketChannel
extends Object
implements AsynchronousChannel, NetworkChannel
```

An asynchronous channel for stream-oriented listening sockets.

An asynchronous server-socket channel is created by invoking the `open` method of this class. A newly-created asynchronous server-socket channel is open but not yet bound. It can be bound to a local address and configured to listen for connections by invoking the `bind` method. Once bound, the `accept` method is used to initiate the accepting of connections to the channel's socket. An attempt to invoke the `accept` method on an unbound channel will cause a `NotYetBoundException` to be thrown.

Channels of this type are safe for use by multiple concurrent threads though at most one accept operation can be outstanding at any time. If a thread initiates an accept operation before a previous accept operation has completed then an `AcceptPendingException` will be thrown.

Socket options are configured using the `setOption` method. Channels of this type support the following options:

Option Name	Description
`SO_RCVBUF`	The size of the socket receive buffer
`SO_REUSEADDR`	Re-use address

Additional (implementation specific) options may also be supported.

Usage Example:

```
final AsynchronousServerSocketChannel listener =
    AsynchronousServerSocketChannel.open().bind(new InetSocketAddress(5000));

listener.accept(null, new CompletionHandler<AsynchronousSocketChannel,Void>() {
    public void completed(AsynchronousSocketChannel ch, Void att) {
        // accept the next connection
        listener.accept(null, this);
```

```
        // handle this connection
        handle(ch);
    }
    public void failed(Throwable exc, Void att) {
        ...
    }
});
```

Since:

1.7

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**AsynchronousServerSocket**(AsynchronousChannelProv	Initializes a new instance of this class.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
abstract Future<AsynchronousSocketCl	**accept**()	Accepts a connection.
abstract <A> void	**accept**(A attachment, CompletionHandler<Asynch super A> handler)	Accepts a connection.
final AsynchronousServerSo	**bind**(SocketAddress local)	Binds the channel's socket to a local address and configures the socket to listen for connections.
abstract AsynchronousServerSo	**bind**(SocketAddress local, int backlog)	Binds the channel's socket to a local address and configures the socket to listen for connections.

abstract SocketAddress	**getLocalAddress**()	Returns the socket address that this channel's socket is bound to.
static AsynchronousServerSo	**open**()	Opens an asynchronous server-socket channel.
static AsynchronousServerSo	**open** (AsynchronousChannelGrou	Opens an asynchronous server-socket channel.
final AsynchronousChannelP	**provider**()	Returns the provider that created this channel.
abstract <T> AsynchronousServ	**setOption** (SocketOption<T> name, T value)	Sets the value of a socket option.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface AsynchronousChannel

close

Methods declared in interface Channel

isOpen

Methods declared in interface NetworkChannel

getOption, supportedOptions

Constructor Details

AsynchronousServerSocketChannel

```
protected AsynchronousServerSocketChannel
(AsynchronousChannelProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider` - The provider that created this channel

Method Details

provider

```
public final AsynchronousChannelProvider provider()
```

Returns the provider that created this channel.

Returns:

The provider that created this channel

open

```
public static AsynchronousServerSocketChannel open
(AsynchronousChannelGroup group)
                                                throws IOException
```

Opens an asynchronous server-socket channel.

The new channel is created by invoking the `openAsynchronousServerSocketChannel` method on the `AsynchronousChannelProvider` object that created the given group. If the group parameter is `null` then the resulting channel is created by the system-wide default provider, and bound to the *default group*.

Parameters:

`group` - The group to which the newly constructed channel should be bound, or `null` for the default group

Returns:

A new asynchronous server socket channel

Throws:

`ShutdownChannelGroupException` - If the channel group is shutdown

`IOException` - If an I/O error occurs

open

```
public static AsynchronousServerSocketChannel open()
                                                throws IOException
```

Opens an asynchronous server-socket channel.

This method returns an asynchronous server socket channel that is bound to the *default group*. This method is equivalent to evaluating the expression:

```
    open((AsynchronousChannelGroup)null);
```

Returns:

A new asynchronous server socket channel

Throws:

`IOException` - If an I/O error occurs

bind

```
public final AsynchronousServerSocketChannel bind
(SocketAddress local)
                                                throws IOException
```

Binds the channel's socket to a local address and configures the socket to listen for connections.

An invocation of this method is equivalent to the following:

```
    bind(local, 0);
```

Specified by:

`bind` in interface `NetworkChannel`

Parameters:

`local` - The local address to bind the socket, or `null` to bind to an automatically assigned socket address

Returns:

This channel

Throws:

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If the channel is closed

`IOException` - If some other I/O error occurs

See Also:

`NetworkChannel.getLocalAddress()`

bind

```
public abstract AsynchronousServerSocketChannel bind
(SocketAddress local,
 int backlog)
                                                  throws IOException
```

Binds the channel's socket to a local address and configures the socket to listen for connections.

This method is used to establish an association between the socket and a local address. Once an association is established then the socket remains bound until the associated channel is closed.

The `backlog` parameter is the maximum number of pending connections on the socket. Its exact semantics are implementation specific. In particular, an implementation may impose a maximum length or may choose to ignore the parameter altogether. If the `backlog` parameter has the value `0`, or a negative value, then an implementation specific default is used.

Parameters:

`local` - The local address to bind the socket, or `null` to bind to an automatically assigned socket address

`backlog` - The maximum number of pending connections

Returns:

This channel

Throws:

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If the channel is closed

`IOException` - If some other I/O error occurs

setOption

```
public abstract <T> AsynchronousServerSocketChannel setOption
(SocketOption<T> name,
 T value)
                                                       throws
IOException
```

Description copied from interface: `NetworkChannel`

Sets the value of a socket option.

Specified by:

setOption in interface NetworkChannel

Type Parameters:

T - The type of the socket option value

Parameters:

name - The socket option

value - The value of the socket option. A value of null may be a valid value for some socket options.

Returns:

This channel

Throws:

IllegalArgumentException - If the value is not a valid value for this socket option

ClosedChannelException - If this channel is closed

IOException - If an I/O error occurs

See Also:

StandardSocketOptions

accept

```
public abstract <A> void accept
(A attachment,
 CompletionHandler<AsynchronousSocketChannel, ? super A> handler)
```

Accepts a connection.

This method initiates an asynchronous operation to accept a connection made to this channel's socket. The handler parameter is a completion handler that is invoked when a connection is accepted (or the operation fails). The result passed to the completion handler is the AsynchronousSocketChannel to the new connection.

When a new connection is accepted then the resulting AsynchronousSocketChannel will be bound to the same AsynchronousChannelGroup as this channel. If the group is shutdown and a connection is accepted, then the connection is closed, and the operation completes with an IOException and cause ShutdownChannelGroupException.

To allow for concurrent handling of new connections, the completion handler is not invoked directly by the initiating thread when a new connection is accepted immediately (see Threading).

Type Parameters:

A - The type of the attachment

Parameters:

`attachment` - The object to attach to the I/O operation; can be `null`

`handler` - The handler for consuming the result

Throws:

`AcceptPendingException` - If an accept operation is already in progress on this channel

`NotYetBoundException` - If this channel's socket has not yet been bound

`ShutdownChannelGroupException` - If the channel group has terminated

accept

```
public abstract Future<AsynchronousSocketChannel> accept()
```

Accepts a connection.

This method initiates an asynchronous operation to accept a connection made to this channel's socket. The method behaves in exactly the same manner as the `accept(Object, CompletionHandler)` method except that instead of specifying a completion handler, this method returns a `Future` representing the pending result. The `Future`'s `get` method returns the `AsynchronousSocketChannel` to the new connection on successful completion.

Returns:

a `Future` object representing the pending result

Throws:

`AcceptPendingException` - If an accept operation is already in progress on this channel

`NotYetBoundException` - If this channel's socket has not yet been bound

getLocalAddress

```
public abstract SocketAddress getLocalAddress()
                                          throws IOException
```

Returns the socket address that this channel's socket is bound to.

Where the channel is `bound` to an Internet Protocol socket address then the return value from this method is of type `InetSocketAddress`.

Specified by:

`getLocalAddress` in interface `NetworkChannel`

Returns:

The `SocketAddress` that the socket is bound to; `null` if the channel's socket is not bound

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.